

A Budget-Gated Reliability Stack for Limit-Aware In-Car Agents: CAR-bench Track 2 with an Open 120B Model

Elias Kohout, Simon Franken, Daniel Brand, Maximilian Stephan

Independent

drift@simon-franken.de

Abstract

We describe our Track 2 submission to the CAR-bench competition: an agent built on the open-weight `gpt-oss-120b` model served by Cerebras. Instead of scaling the model, we spend the track’s inference budget on *reliability*. A three-stage pipeline — constrained generation with deterministic enforcement retries, a risk-gated LLM verifier, and a bounded revision — is hard-capped in code at the track limit of five sequential LLM calls per baseline step, with the cap enforced by a shared per-step budget rather than by convention. Three deterministic guards add reliability at *zero* additional calls: phase separation (gather before acting), literal argument traceability, and a capability-catalog diff that tells the agent which ordinarily available tools are absent from the current session. On the public test split our agent reaches 58.7% Pass³, up from 43.3% for our prompt-only baseline, placing an open 120B model in the range reported for substantially larger proprietary systems. We report ablations, four techniques that *failed*, and the measurement noise floor that makes such comparisons fragile.

1 Introduction

CAR-bench evaluates whether an in-car assistant stays *consistent* and *limit-aware* under real-world uncertainty [4]. Its three task families probe distinct failure modes: `BASE` measures ordinary task completion, `HALLUCINATION` removes tools from the session and checks whether the agent admits the limitation instead of fabricating a result, and `DISAMBIGUATION` checks whether ambiguity is resolved from context or, when it cannot be, surfaced to the user. Scoring is strict: a task counts only if tool use, final state, intermediate state, and policy compliance are all correct, and we report Pass³ — the consistency metric introduced with τ -bench [9], under which all three trials of a task must succeed — which punishes non-determinism as hard as outright error. Agent and evaluator communicate over the Agent2Agent (A2A) protocol [1].

Track 2 constrains inference rather than model choice: at most five *sequential* LLM calls per baseline step (parallel

calls inside a step are permitted) and at most 500k tokens per task on average. The reference baseline uses one call per step. This framing invites a specific question, which is the subject of this report: *given a mid-sized open model, what is the most score-efficient way to spend four additional calls?*

Our answer is that most of the gain does not come from the extra calls at all. The largest single improvement in our system is a deterministic, zero-call prompt injection; the second largest is a deterministic validator. The extra LLM calls are worth spending only when they are *gated* — fired on a minority of steps that heuristics flag as risky. We contribute: (i) a per-step call budget that makes the track constraint a code invariant (Section 2); (ii) a set of deterministic guards derived from CAR-bench’s own error taxonomy (Section 3); (iii) an evaluation showing 43.3% $\rightarrow$ 58.7% Pass³ with a per-technique breakdown (Section 4); and (iv) an honest account of four techniques that did not survive validation, plus the noise floor that nearly hid that fact (Section 5).

2 Architecture and Call-Budget Compliance

2.1 Unit of accounting

We define one *baseline LLM step* as one action generation: one A2A round in which the agent receives a message and returns exactly one action (tool calls or a user-facing response). This is the unit the reference baseline serves with a single LLM call, and it corresponds one-to-one to an invocation of `execute()` in our agent. The budget is allocated and reset per invocation, so the constraint holds per step by construction.

We stress this because the A2A token-reporting field `turn_metrics.num_llm_calls` aggregates over a whole conversational *turn* (several rounds of tool calling before the agent finally speaks) and therefore legitimately exceeds five. Auditors should read our per-step log line `sequential_llm_calls.this_step`, which is emitted once per baseline step.

2.2 Pipeline

Figure 1 shows the pipeline. Stage ① generates a next action under a JSON response schema [8] that forces the model to fill a `checks` object (`missing_capability`, `unspecified_value_source`, `scope_ok`) *before* naming an action, which acts as a short structured chain of

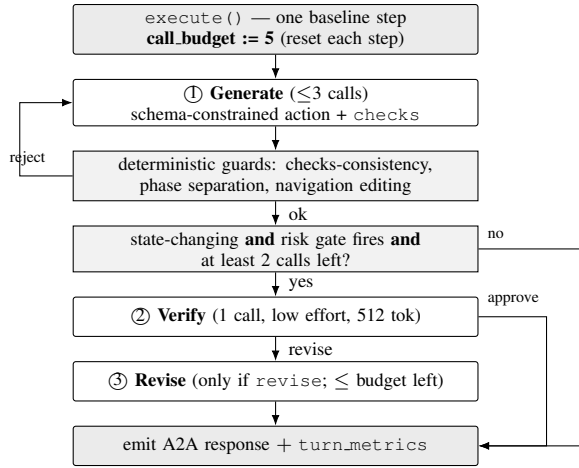


Figure 1: One baseline LLM step. **Maximum sequential depth: 5** (3 + 1 + 1, boxed stages are the only LLM calls); common case 1–3. **Parallel fan-out: 1** — no stage issues concurrent calls, so nothing is exempt from the sequential count. Branches are the two gates. There is one retry loop, the enforcement loop inside stage ①; stage ③ re-enters it with the remaining budget, and its iterations are counted against the same budget in both places.

thought. The parsed action then passes three deterministic validators; a violation raises a malformed-response error that triggers a retry carrying a targeted correction, but never on the final permitted attempt — an inconsistent action is preferable to a turn error.

Stage ② is an LLM verifier — the generate–critique–revise loop of self-refinement [5; 7] with an LLM judge [10] as the critic, but deliberately narrow: it runs only on state-changing actions, uses low reasoning effort and a 512 token cap, and returns *approve* or *revise* with problems. A risk gate suppresses it entirely in the common, uninteresting case — a single state-changing call whose argument values are all literally traceable to the conversation. Stage ③ is a single bounded revision seeded with the verifier’s problems. Neither stage can fail a turn: on any error the stage ① action stands.

2.3 Why the bound holds

Table 1 accounts for the worst case. The key mechanism is that every stage receives the *remaining* budget, not a private allowance: stage ① caps its attempts at $\min(\text{retries} + 1, \text{budget})$; the verifier runs only if at least two calls remain (so that a revision it requests is always affordable); the revision receives exactly what is left, re-entering stage ①’s enforcement loop under that reduced cap, so the split is not fixed (1+1+3 is as admissible as 3+1+1). An optional completeness pass exists in the code but is disabled in the submitted configuration; it shares the same guard, so the bound is independent of that toggle. Retries are counted as calls, which is conservative — a lenient reading would exclude them.

3 Reliability Techniques

Zero-call deterministic guards. Three validators inspect the parsed action and, on violation, force a corrected regen-

	Stage	Max	Total
①	Generate (1 + 2 enforcement retries)	3	3
②	Verify (gated: needs ≥ 2 left)	1	4
③	Revise (budget — used; re-enters ①)	1	5

Table 1: Worst-case sequential calls per baseline step; the “Max” column is the value each stage can take *in this path*, and every other path is bounded by the same total. Measured average token use in the submitted configuration is $\sim 92\text{k}$ per task against the 500k limit.

eration within stage ①. *Checks-consistency* rejects actions that contradict the model’s own self-report (e.g. declaring a missing capability and then changing state anyway). *Phase separation* implements the recommendation from the CAR-bench analysis to separate information gathering from execution: no state change without a prior read-only call, and no clarifying question without a tool attempt since the last user message. *Navigation editing* forces the agent to observe navigation state before replacing an active route — a rule the scoring makes cheap to enforce, since intermediate-state credit is a *subset* check over state hashes: additional read-only calls carry no penalty, while wrong state changes do.

Argument-level anti-hallucination. Every argument of a state-changing call must appear literally in the conversation (user utterance or tool result); values that are invented or silently defaulted are rejected. A separate validator catches template placeholders ($\{\{...\}\}$, $\langle...\rangle$, `tbid`) that the model emits when chaining dependent calls. The configuration that adds this second validator — together with the toll notice below, which we did not measure separately — moved BASE from 62% to 66–72%, mostly on multi-step route lookups.

Capability-catalog notice. The agent ships a static catalog of the tools and parameters that are ordinarily present in this domain, compiled from the public training split. At prompt-construction time it diffs the catalog against the tools the evaluator actually supplied and, if anything is missing, injects a notice naming the gap and instructing the agent to say plainly that it cannot do that part — and not to ask the user to supply it, claim success, or fake a workaround. This is a static prompt prior, not runtime introspection: the diff consumes only evaluator-provided tool definitions, and the resulting behaviour stays fully visible to the hallucination metric. It is our single largest win (HALLUCINATION 42% $\rightarrow$ 52–54%, twice confirmed) and fired zero times across all 75 BASE and DIS-AMBIGUATION tasks, i.e. it is inert where it should be. Two smaller injections follow the same pattern: a toll-disclosure reminder when a retrieved route sets `includes_toll`, and a notice not to re-ask for information the user has just declined to provide.

Gated verification. Instrumentation of the best run quantifies both the gate and the reviewer. Of 338 steps that reached the gate, 238 (70%) were skipped outright; the verifier ran 99 times and requested a revision in 30 of them (30%). The confirmation run reproduces this profile closely (223 skipped, 87 reviews, 25 revisions). Inspection of a sample showed substantive catches — a policy violation, invented values, unre-

Configuration	Base	Hall.	Dis.	Pass ³
Prompt-only baseline	60	42	28	43.3
+ guards, gated verifier	64	42	48	51.3
+ catalog notice	62	54	40	52.0
+ placeholder, toll	66	58	52	58.7
<i>confirmation run</i>	72	58	40	56.7

Table 2: Pass³ (%) on the public test split, 125 tasks $\times$ 3 trials, temperature 0. Rows are cumulative configurations, each measured in a separate full run. The shared budget of Section 2.3 was added to the code after these runs; a per-step audit of their logs finds at most five sequential calls everywhere except one step of the 58.7% run, which used six (0.05% of its 1953 steps). The confirmation run is within the bound throughout.

requested extra actions — rather than stylistic nagging. One verifier call failed outright and, as designed, left the stage ① action untouched instead of failing the turn. Gating keeps this affordable: the full stack spends 92k tokens per task against 85k for the prompt-only baseline, an overhead of +9% (confirmation run: +5%).

4 Results

Table 2 reports full runs on the public test split (125 tasks, three trials, temperature 0; Pass³ is averaged over the three task families). Every row is a separate full evaluation. Reaching the final configuration took 16 such runs — median 75 minutes of wall-clock time each, with provider throttling stretching four of them past two hours — alongside 27 shorter runs on task subsets, over eight days. We report full-run numbers exclusively: as Section 5 shows, the short runs we used for iteration were systematically misleading, and several configurations that looked best on a subset lost on the full split. The stack raises Pass³ from 43.3% to 58.7%, with the best configuration confirmed twice in the 56–59% range. For context, the baseline table published with CAR-bench [4] places the same open model at 28% and reports 41% for Gemini-2.5-flash (thinking) and 54% for GPT-5 (thinking); our figures are not directly comparable (different split conditions and user-simulator routing), but the gap between 28% and 58.7% for one and the same open model is the result we consider meaningful.

The per-family pattern is informative. DISAMBIGUATION responds to structural interventions (phase separation fired 170 times in one run and lifted the family 28 $\rightarrow$ 48), whereas HALLUCINATION responds almost exclusively to the catalog notice. The two families also interfere: a preference-consultation rule that solved several DISAMBIGUATION tasks eroded HALLUCINATION consistency, an instance of the completion-versus-compliance tension the CAR-bench authors describe.

5 Negative Results and Measurement Noise

We report four techniques that looked promising and did not survive. (i) A *workflow memory* mined from the training split (canonical tool prerequisites, 312 tokens) scored 73% on a focused slice but 48.0% on a full run versus 51.3%; the

prime suspect is a “let the user choose” phrasing that induced asking in tasks requiring internal resolution. (ii) A *response scaffold* forcing an explicit cannot-statement plus fallback offer produced the first-ever full flips on our stubborn missing-response class in focused runs, yet that class flipped back at the run level despite 88 activations; after seven attempts we closed it as a model-capability limit. (iii) A *completeness verifier* raised Pass@3 to 80% while Pass³ fell from 52.0% to 44.7%: 109 self-second-guessing revisions turned thirteen stable tasks flaky — a reminder that under Pass³, variance is a first-class cost, and consistent with reports that models do not reliably self-correct without an external signal [3]. (iv) An *enrichment rule* for user preferences gained one task and broke our most stable marker task.

These conclusions are only possible because we first measured the noise floor. At temperature 0 the three trials of a run are near-clones, so borderline tasks flip at the *run* level rather than the trial level; an identical configuration measured twice on a 15-task slice produced 66.7% and 46.7%. We therefore accept a change only when it is deterministic (a $0/x \rightarrow x/x$ flip) or reproduces across at least two independent full runs, and we treat ± 8 pp on DISAMBIGUATION as noise. A second, non-obvious confound: the evaluator’s user simulator is itself an LLM, and provider congestion truncated its JSON badly enough to produce a 18.7% run that had nothing to do with agent quality. Run-level evaluator error counts correlate with score and must be checked before any comparison.

Where the remaining errors live. To decide when to stop, we classified the deterministically failing tasks of our best run by root cause. Only about four of thirty were cleanly addressable by harness work. The remainder split into route or destination selection (5 tasks, a reasoning failure), under-acting and omitted actions (4, semantics), final state diverging from ground truth (4, reasoning), navigation state that is simply not observable unless the agent chooses to look (5), and content or formatting requirements such as toll disclosure and 24-hour times. This distribution is why we stopped adding mechanisms: the harness can enforce structure and surface facts, but it cannot supply reasoning the underlying model does not have. It also explains the asymmetry in Table 2 — interventions that convert an *observability* gap into an explicit prompt fact (the catalog notice, the navigation-state guard) pay off immediately, whereas the reasoning-bound classes barely move.

6 Compliance and Reproducibility

The submitted agent uses Cerebras-hosted [2] gpt-oss-120b [6] through the Cerebras SDK, with the model name, service tier, reasoning-effort selectors, sampling parameters and every toggle of Table 3 exposed as environment variables in the submitted `scenario.toml`; the client pins the Cerebras production endpoint, so the organizers need only supply the API key. The agent uses exclusively evaluator-provided inputs — system and policy text, transcript, tool definitions, tool results — plus the static public-domain catalog described in Section 3. It does not execute CAR-bench tools, read hidden task state or answer keys, or add private tools to the decision loop, and

Enabled	MAX_CALLS_PER_STEP=5, VERIFIER, VERIFIER_GATE, ENFORCE_CHECKS, PHASE_SEPARATION, NAV_EDITING, CATALOG_NOTICE, TOLL_NOTICE
Disabled	COMPLETENESS, WORKFLOWS, RESPONSE_SCAFFOLD
Sampling	EXECUTOR_MODEL=gpt-oss-120b, EXECUTOR_REASONING_EFFORT=medium, VERIFIER_REASONING_EFFORT=low, TEMPERATURE=0, MAX_COMPLETION_TOKENS=2048, LLM_MALFORMED_RETRIES=2

Table 3: Submitted configuration. Every name carries the prefix `TRACK2_` and is an environment variable whose default in the submitted `scenario.toml` is the value shown, so the configuration is reproduced even when nothing is injected but the API key.

it reports `prompt_tokens`, `completion_tokens` and `thinking_tokens` on final responses. The image is public and digest-pinned; it was validated end-to-end against the official evaluator image via the repository’s GHCR validation path. Table 3 lists the exact submitted configuration.

Which code produced which number. The submitted image carries the budget-enforced agent, whose sources are identical to those of one further full run made after the guard landed; that run scored 47.3% Pass³ (60/42/40). We report it rather than hide it, but it does not measure a cost of the cap: the guard never bound in that run (zero budget-limited skips), and its per-step call profile is the one the earlier 56.7% run already had, which stayed within five calls throughout. The difference is run-level variance of the kind quantified in Section 5, which is also why we state the 56–59% band rather than the single best number.

Assumptions behind the compute figures. Token counts are what the agent reports through A2A: prompt, completion and thinking tokens summed over *all* internal LLM calls of a turn, including calls that were discarded by an enforcement retry, and including the verifier and revision calls. They are therefore an upper bound on what a lenient accounting would report. The ~92k figure is the mean over the 373 of 375 task-trials of the best run that returned token counts (two failed with an evaluator error and report zero); the median is 82k and the single most expensive trial reached 584k, so the average sits comfortably under the 500k limit while individual pathological conversations can exceed it. Wall-clock latency is *not* a meaningful basis for *comparing* configurations in our data: runs were executed across several days on a shared provider tier, and between-day variation in provider load exceeded the effect of our configuration changes. The absolute figures are still informative, because the evaluator meters agent-side LLM time itself: the best run spends 4.3s of LLM time per task-trial (median 3.5s) and 0.87s per A2A round, against 5.2s and 1.05s for the prompt-only baseline, and the confirmation run reproduces this (4.3s, 0.91s). At this reasoning effort the gated stack therefore buys its reliability without a latency penalty. We nevertheless quote token overhead rather than latency overhead as our cost measure

throughout.

Two caveats belong on the record. First, the capability catalog of Section 3 is prior knowledge: it is compiled offline from the *public* training split and shipped as a static file. It contains no task, answer or hidden-state information and is never updated at run time, but it does encode the expectation that a session normally offers a particular set of tools. Second, our development numbers were produced with the evaluator’s user simulator and policy judge routed through a third-party provider rather than the organizers’ configuration; scores are therefore internally comparable but not directly comparable to published leaderboard figures.

7 Limitations and Future Work

Our development loop iterated on a fixed public split, and we measured the resulting selection bias directly: at an early stage of development, tasks we had been tuning against scored roughly 70% while unseen tasks of the same run scored about 43%, with unseen DISAMBIGUATION as low as 13%. We reacted by validating only on full runs and by preferring mechanisms that are deterministic and domain-general — a guard that enforces “observe before replacing an active route” encodes a domain rule, not a task — but some adaptation to the public split certainly remains, and the hidden-set result should be expected to fall below our development figure.

Three directions remain open. First, the verifier currently checks several aspects in one pass; splitting it into narrow single-aspect reviewers is affordable within the call budget and may sharpen the precision of its 30% revision rate. Second, best-of- N sampling is attractive under Pass³ but interacts badly with provider rate windows, so it needs a scheduling story before it can be evaluated fairly. Third, our workflow-memory experiment failed for what looks like a single phrasing choice rather than a conceptual reason; mined canonical tool orderings deserve a second attempt with the offending route-selection line removed and DISAMBIGUATION validated separately.

8 Conclusion

Spending a fixed inference budget on *gated* verification and *deterministic* guards, rather than on uniformly deeper reasoning, lifted an open 120B model by more than 15 points of Pass³ on CAR-bench. The most transferable lesson is that the cheapest interventions dominated: a static capability diff and an argument validator, both costing no additional LLM calls, account for the majority of the gain, while the one technique that added *ungated* model calls actively hurt by injecting variance into a metric that punishes it. Limit-awareness, in our experience, depends less on how well the model reasons than on whether the relevant constraint is placed in front of it, and checked mechanically, at the moment it acts.

References

- [1] A2A Project. Agent2Agent (A2A) protocol, version 1.0. The Linux Foundation, 2026. <https://a2a-protocol.org/v1.0.0/>.

- [2] Cerebras Systems. Cerebras inference: wafer-scale inference API. <https://inference.cerebras.ai/>, 2026.
- [3] Jie Huang, Xinyun Chen, Swaroop Mishra, et al. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, 2026. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.1886, <https://aclanthology.org/2026.acl-long.1886/>.
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [6] OpenAI. gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925, 2025.
- [7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.
- [9] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024.
- [10] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.